RUNBOOK — Captain Adel SaaS (accounts, billing & quota on captadel.com)

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-06-16

> **Note (2026-06-13):** Captain Adel now lives in its own repo, `FlyGACA/Captain-Adel`. The code referenced as `captadel/…` below now sits at that repo's root.

Stand up the pilot-subscription layer on captadel.com: a **separate Captadel Firebase project** for accounts, Stripe for billing, and a free daily quota with a Pro unlimited tier. The code is already in `captadel/` and **ships dark** — with none of the steps below done, the site is exactly as it was (free for everyone, no sign-in, no paywall). Each step lights up one piece.

> **PDPL (load-bearing).** Real user questions and accounts are personal data and must be processed in-Kingdom. Put Firestore in **me-central2 (Dammam)** and keep the Cloud Run service in a KSA region (see `runbook-captadel-deploy.md`).

> **Architecture.** Billing lives **inside the Express service** (`captadel/src/billing/`), not in Cloud Functions. The browser only ever talks to Firebase **Auth**; plan and quota come from the service's `GET /v1/me` (Admin SDK). `firestore.rules` denies every direct client read/write. Deploy the Cloud Run service into the **same GCP project** as the Firebase project so `firebase-admin` uses ADC with zero config.

0. Where things are

Piece	Path
Billing routes (checkout/webhook/portal/me/config)	`captadel/src/billing/routes.js`
Entitlement writer + pure cores	`captadel/src/billing/entitlements*.js`, `stripe-core.js`, `tier-core.js`
Quota (Firestore) + calendar math	`captadel/src/quota/quota.js`, `quota-core.js`
Admin SDK singleton	`captadel/src/firebase.js`
Caller identity middleware (60s cache)	`captadel/src/middleware/auth.js`
Client auth / billing / account	`captadel/public/assets/js/{auth,billing,account}.js`
Web config (fill in step 1)	`captadel/public/assets/js/firebase-config.js`

Piece	Path
Pricing section	`captadel/public/index.html` (`#pricing`), account page `account.html`
Firestore project files	`captadel/firebase.json` , `firestore.rules` , `.firebaserc`
Deploy	`captadel/deploy/deploy.sh` , `.github/workflows/deploy-captadel.yml`

---

1. Create the Captadel Firebase project

This is a **new project**, separate from Fly GACA's `flygaca-app` — captadel.com keeps its own user base and billing.

1. Firebase console → **Add project** (e.g. `captadel-app`). Update the id in `captadel/.firebaserc` if you choose a different one.
2. **Firestore Database** → Create → **Location** `me-central2` (PDPL — cannot be changed later). Production mode.
3. **Authentication** → Sign-in method → enable **Email/Password** and **Google**.
4. **Authentication → Settings → Authorized domains** → add `captadel.com` , `www.captadel.com` (and your Cloud Run `*.run.app` host for testing).
5. **Project settings → Your apps → Web app** → register an app, copy the config into `captadel/public/assets/js/firebase-config.js` (replace the `REPLACE_WITH_*` placeholders). The web API key is public by design.
6. **Firestore → TTL** → create a policy on collection `adelQuota` , field `expireAt` , so spent quota counters self-purge.

Commit the filled `firebase-config.js` . Once the placeholders are gone, the account page shows real sign-in instead of the "accounts open soon" state.

Deploy the deny-all rules

```
cd captadel
firebase deploy --only firestore:rules --project captadel-app
```

---

2. Point the Cloud Run service at this project

`firebase-admin` uses ADC, so the service must run **in the same GCP project** as the Firebase project above.

```
gcloud config set project captadel-app
cd captadel && ./deploy/deploy.sh secrets     # re-create model secrets here
./deploy/deploy.sh                            # deploy (me-central2; me-central1
fallback)
```

Then map `captadel.com` (see `runbook-captadel-deploy.md §5`) and, if the service URL changed, update `ADEL_API_URL` on the Fly GACA gateway so the embedded flygaca.com chat still reaches the same brain.

> The service auto-detects the project from `GOOGLE_CLOUD_PROJECT` on Cloud Run. For local dev: `export FIREBASE_PROJECT_ID=captadel-app` and `gcloud auth application-default login`.

---

3. Stripe

1. Create a product **"Captain Adel Pro"** with two recurring prices:
   - **Annual** — SAR 299/yr (founding). (Regular SAR 349/yr — raise when ready.)
   - **Monthly** — SAR 35/mo. The static pricing copy in `index.html` must match the dashboard numbers.
2. Copy the price IDs and the secret key into Secret Manager:

   ```
   printf '%s' "sk_live_…"   | gcloud secrets create STRIPE_SECRET_KEY     --data-
   file=-
   printf '%s' "price_…ann"  | gcloud secrets create STRIPE_PRICE_ANNUAL   --data-
   file=-
   printf '%s' "price_…mon"  | gcloud secrets create STRIPE_PRICE_MONTHLY  --data-
   file=-
   ```

3. **Webhook**: Stripe dashboard → Developers → Webhooks → add endpoint `https://captadel.com/v1/billing/webhook`, events: `checkout.session.completed`, `customer.subscription.created`, `customer.subscription.updated`, `customer.subscription.deleted`. Copy the signing secret:

   ```
   printf '%s' "whsec_…" | gcloud secrets create STRIPE_WEBHOOK_SECRET --data-file=-
   ```

4. **Customer portal**: Stripe → Settings → Billing → Customer portal → activate (lets pilots cancel/manage from `account.html`).
5. Re-run `./deploy/deploy.sh` so the new secrets are wired. Checkout returns 503 until both `STRIPE_SECRET_KEY` and the project are present.

---

4. GitHub Actions deploy

The active workflow is `.github/workflows/deploy-captadel.yml` (push to main, paths `captadel/**`; or manual `workflow_dispatch`). It is inert until configured.

1. Create a deployer service account in the Captadel project with roles `roles/run.admin`, `roles/cloudbuild.builds.editor`, `roles/iam.serviceAccountUser`, `roles/secretmanager.secretAccessor`. Mint a JSON key.
2. Repo → Settings → Secrets and variables → Actions:
   - **Secret** `GCP_SA_KEY` = the JSON key.
   - **Variables** `CAPTADEL_PROJECT_ID` = `captadel-app`, `CAPTADEL_REGION` = `me-central2` (or `me-central1`).
3. Push to main (touching `captadel/**`) or run the workflow manually. The health check asserts `/health` returns `status:ok`.

> **Upgrade path (keyless):** swap the SA key for Workload Identity Federation — add `permissions: { id-token: write }` and use `google-github-actions/auth@v2` with `workload_identity_provider` + `service_account`. Remove `GCP_SA_KEY` once WIF is verified.

---

5. Launch sequence

The layer ships **dark**. Bring it up deliberately:

1. **Dark (default):** deploy with `ADEL_LAUNCH_MODE=free`. Everyone is unmetered; sign-in works (once step 1 is done) but is optional; checkout 503s until Stripe is wired. Good for soft-launching accounts without a paywall.
2. **Billing test:** in Stripe **test mode**, run a checkout with card `4242 4242 4242 4242`; confirm `users/{uid}.entitlement.plan === 'pro'` and the account page shows the Pro badge. Cancel in the dashboard → entitlement drops to free.
3. **Go live:** unset `ADEL_LAUNCH_MODE`, set the allowances, redeploy:

   ```
   ADEL_DAILY_FREE=5 ADEL_DAILY_ANON=5 ADEL_FREE_PERIOD=day \
   SITE_URL=https://captadel.com ./deploy/deploy.sh
   ```

   A free signed-in pilot now gets 5 cited questions/day; the 6th returns a 402 with the bilingual upgrade nudge; Pro is unlimited (abuse limiter still on).

---

6. Rollback

- **Instant "everything free":** set `ADEL_LAUNCH_MODE=free` and redeploy — the quota goes dormant immediately, no code change.

- **Code:** roll back the Cloud Run revision (`gcloud run services update-traffic captadel --to-revisions REV=100 …`).
- **Billing only:** remove the `STRIPE_*` secrets and redeploy — checkout 503s and the site reverts to its free-during-launch state; existing entitlements remain.

---

7. Verify (local)

```
cd captadel
npm run test:unit        # cores + the raw-body webhook signature test
npm run smoke            # loads with zero billing env (proves dark-launch safety)

# Live billing loop (Stripe test mode):
export FIREBASE_PROJECT_ID=captadel-app
gcloud auth application-default login
export STRIPE_SECRET_KEY=sk_test_…  STRIPE_WEBHOOK_SECRET=whsec_…  \
       STRIPE_PRICE_ANNUAL=price_…  SITE_URL=http://localhost:8787
npm start &
stripe listen --forward-to localhost:8787/v1/billing/webhook
# → sign up on /account.html, checkout with 4242, watch the entitlement flip.

# Gateway contract unchanged (flygaca embed):
curl -s -XPOST localhost:8787/v1/chat -H 'X-Adel-Api-Key: …' \
  -H 'Content-Type: application/json' -d '{"message":"hi","product":"flygaca"}'
```